

TD 3 & TD4
L'héritage et les classes abstraites

Exercice 1/

Soit les deux classes `Personne` et `Compte` du fichier `ComptePersonne.java`.

Question 1/ Que décrivent-elles ? De quelle nature est la variable d'instance `titulaire` dans la classe `Compte` ?

Complétez la classe `Compte` en écrivant :

- une méthode `public String toString()`
- une méthode `void afficher()`

Dans une classe `Td4_Banque`, écrivez un `main()` qui crée quelques personnes, puis les affiche.

Question 2/ Complétez la classe `Compte` en écrivant :

- une méthode `void afficher()`

Testez la création d'un compte et son affichage.

Question 3/ Complétez la classe `Compte` en écrivant :

- une méthode pour déposer une somme d'argent
- une méthode pour retirer une somme du compte (le compte ne doit pas être débiteur après le retrait et si le compte n'est pas suffisamment approvisionné, le retrait est refusé).

Complétez le `main()` avec la création de quelques comptes, le dépôt et le retrait de quelques sommes, leur affichage etc.

Question 4/ Ecrivez une classe `CompteRemunere` qui hérite de la classe `Compte`. Un compte rémunéré est un compte bancaire caractérisé, en plus, par un taux de rémunération (`double`). Ecrivez un constructeur qui reçoit en paramètres toutes les données nécessaires.

Peut-on retirer ou déposer un montant sur une instance de `CompteRemunere` ? Peut-on appliquer les méthodes `afficher()` et `toString()` de la classe `Compte` à une instance de la classe `CompteRemunere` ? Que se passe-t-il ? Définissez les méthodes `public String toString()` et `void afficher()` de la classe `CompteRemunere`.

Question 5/ Prévoir une méthode `void versementInt()` qui verse les intérêts sur le solde d'un compte rémunéré (on rajoute le solde multiplié par le taux d'intérêt au solde). Peut-on la définir dans la classe `Compte` ? Dans quelle classe la définir ?

Question 6/ Parmi les instances de `Compte` on peut considérer des comptes particuliers, les Comptes d'entreprise qui sont munis d'une variable d'instance en plus qui est l'entreprise qui en est propriétaire. Une instance d'`Entreprise` aura deux variables d'instance : son nom (`String`) et le comptable (`Personne`) et une méthode `public String toString()` ainsi qu'une méthode d'affichage.

Ecrivez deux classes, celles des `Entreprises` et celle des `CompteDentreprise`. La classe `CompteDentreprise` peut-elle hériter de la classe `Compte` ? Comment se débrouiller ? Prévoir l'affichage d'un compte d'entreprise et une méthode `toString()`.

Question 7/ Définissez une méthode `void mettreSoldeAzero()` qui met le solde d'un compte à 0. Attention, cette méthode ne devra être possible que pour les comptes d'entreprise.

Question 8/ Dessinez l'arbre d'héritage que vous avez obtenu.

Question 9/ Créez une classe `Banque` ayant en variables d'instance un nom (`String`) et un tableau de `Compte` et les méthodes pour :

- ajouter un compte
- afficher la banque (son nom et tous ses comptes)

Pensez-vous pouvoir rajouter dans la banque des comptes d'entreprises et des comptes rémunérés ? Si vous modifiez un compte de la banque (par un dépôt par exemple), est-ce que l'affichage de la banque s'en trouve modifié ? Est-ce un avantage ou un inconvénient ?

Exercice 2/

On développe pour un zoo une application de réalité augmentée. Lorsque qu'un animal passe près d'une des caméras qui filment l'ensemble des enclos, son image est entourée sur un écran et son espèce ainsi que les informations qui y sont rattachées y sont affichées.

Question 1/ Créer une classe *Animal* puis les classes *Âne*, *Chouette* et *Lion* qui en héritent. Chaque animal sera identifié par un numéro Pourquoi faut-il que la classe *Animal* soit abstraite ?

Question 2/ Destinée à des enfants, cette application affiche le type de cri que pousse l'animal, ainsi la chouette hulule, le lion rugit et l'âne brait. Au lieu de stocker cette information à l'aide d'une variable d'instance, on décide de définir une méthode d'instance abstraite `void cri()` au niveau de la classe `Animal` qui affiche sur la console le cri de chaque bête. Vous spécifierez par la suite son comportement dans chacune des sous-classes concrètes.
